

Algoritmos de Aprendizado

- Regra de Hebb
- Perceptron
- Delta Rule (Least Mean Square)
- Multi-Layer Perceptrons (Back Propagation)
- Hopfield
- Competitive Learning
- Radial Basis Functions (RBFs)

Multi-Layer Perceptrons

- Redes de apenas **uma camada** só representam funções **linearmente separáveis**
- Redes de **múltiplas camadas** solucionam essa restrição
- O desenvolvimento do algoritmo **Back Propagation** foi um dos motivos para o **ressurgimento** da área de **Redes Neurais**

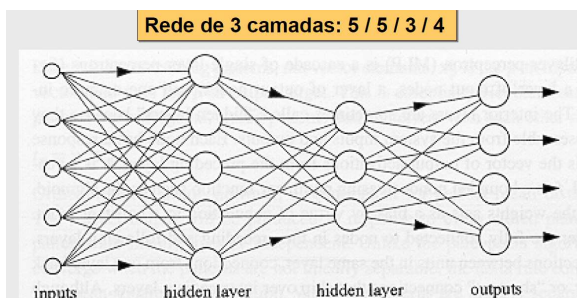
Multi-Layer Perceptrons

- Redes de apenas **uma camada** só representam funções **linearmente separáveis**
- Redes de **múltiplas camadas** solucionam essa restrição

Back Propagation

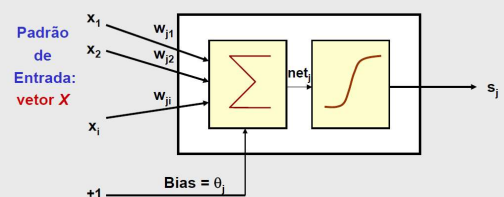
- O grande **desafio** foi achar um algoritmo de aprendizado para a **atualização dos pesos** das camadas intermediárias.
- **Idéia Central:**
 - Os **erros** dos elementos processadores da **camada de saída** (conhecidos pelo **treinamento supervisionado**) são **retro-propagados** para as **camadas intermediárias**

Multi-Layer Perceptrons



Back Propagation

Modelo básico do Neurônio Artificial:



Back Propagation

• Características Básicas:

- Regra de Propagação → $net_j = \sum x_i \cdot w_{ji} + \theta_j$
- Função de Ativação → **Função Não-Linear, diferenciável em todos os pontos.**
- Topologia → **Múltiplas camadas.**
- Algoritmo de Aprendizado → **Supervisionado**
- Valores de Entrada/Saída → **Binários e/ou Contínuos**

Processo de Aprendizado

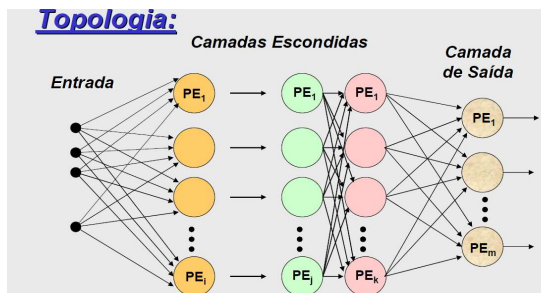
- ↻ Deve-se **minimizar** o erro de **todos os processadores** da camada de saída, para **todos os padrões**



Usa-se o E_{SSE} → **Sum of Squared Errors**

Back Propagation

Topologia:



Processo de Aprendizado

E_{SSE} → **Sum of Squared Errors**

- $E_{SSE} = \frac{1}{2} \sum_p \sum_j (t_{pj} - s_{pj})^2$
- t_{pj} = valor desejado de saída do padrão p para o processador j da camada de saída
- s_{pj} = estado de ativação do processador j da camada de saída quando apresentado o padrão p

Processo de Aprendizado

- Processo de **minimização** do erro quadrático pelo método do **Gradiente Decrescente** → $\Delta w_{ji} = -\eta \frac{\delta E}{\delta w_{ji}}$
- Cada **peso sináptico i** do elemento processador **j** é atualizado proporcionalmente ao **negativo da derivada parcial do erro** deste processador com relação ao peso.

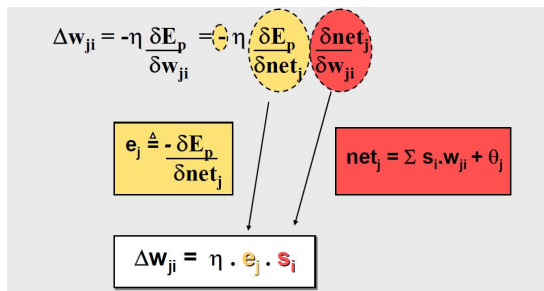
Processo de Aprendizado

E_{SSE} → **Sum of Squared Errors**

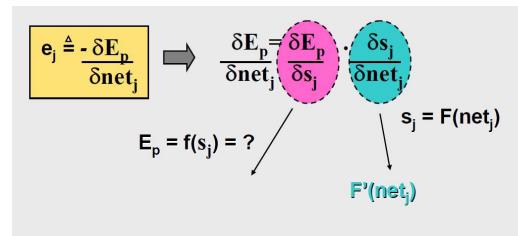


Vantagem: os erros para diferentes padrões, para diferentes saídas são **INDEPENDENTES**

Cálculo de Δw_{ji}

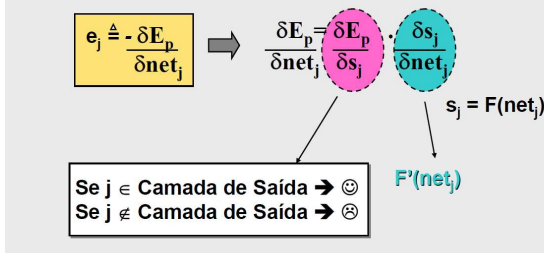


Cálculo do Erro (e_j) $j \in \text{Camada Escondida}$



Cálculo do Erro (e_j)

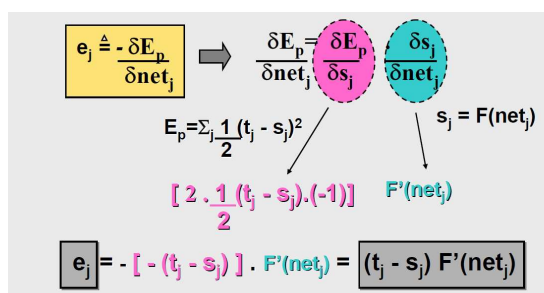
Depende da camada a qual o processador j pertence:



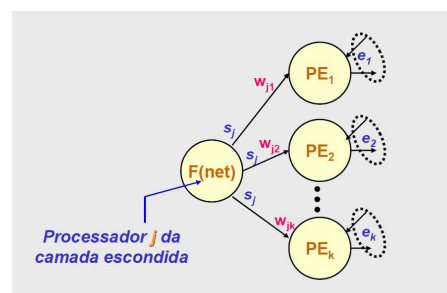
Cálculo do Erro (e_j) $j \in \text{Camada Escondida}$

- Pelo **aprendizado supervisionado**, só se conhece o erro na **camada de saída** (e_k);
- Erro na saída (e_k) é função do **potencial interno** do processador (net_k);
- O net_k depende dos **estados de ativação** dos processadores da camada anterior (s_j) e dos pesos das conexões (w_{kj});
- Portanto, s_j de uma camada escondida afeta, em maior ou menor grau, o erro de todos os processadores da camada subsequente.

Cálculo do Erro (e_j) $j \in \text{Camada de Saída}$



Cálculo do Erro (e_j) $j \in \text{Camada Escondida}$



Cálculo do Erro (e_j) $j \in \text{Camada Escondida}$

$$e_j \triangleq - \frac{\delta E_p}{\delta \text{net}_j} \Rightarrow \frac{\delta E_p}{\delta \text{net}_j} = \frac{\delta E_p}{\delta s_j} \cdot \frac{\delta s_j}{\delta \text{net}_j}$$

$$\frac{\delta E_p}{\delta s_j} = \frac{\delta}{\delta s_j} \left[\frac{1}{2} \sum_k (t_k - s_k)^2 \right]$$

$$= \frac{1}{2} \left[\sum_k (t_k - s_k) \right] \cdot (-1) \cdot \frac{\delta s_k}{\delta s_j}$$

$$= - \left[\sum_k (t_k - s_k) \right] \cdot F'(\text{net}_k) \cdot \frac{\delta \text{net}_k}{\delta s_j}$$

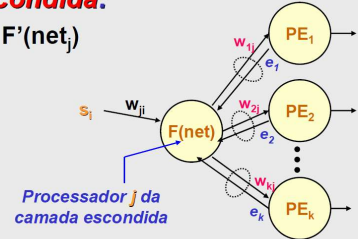
$$e_j = - \left\{ \sum_k e_k \cdot w_{kj} \right\} \cdot F'(\text{net}_j)$$

$$e_j = F'(\text{net}_j) \cdot \sum_k e_k \cdot w_{kj}$$

Cálculo do Erro (e_j)

❶ **Processador j pertence à Camada Escondida:**

$$e_j = (\sum e_k \cdot w_{kj}) \cdot F'(\text{net}_j)$$



Processo de Aprendizado

- Em resumo, após o cálculo da derivada, tem-se:

$$-\Delta w_{ji} = \eta \cdot s_i \cdot e_j$$

Onde:

- s_i → valor de entrada recebido pela conexão i
- e_j → valor calculado do erro do processador j

Processo de Aprendizado

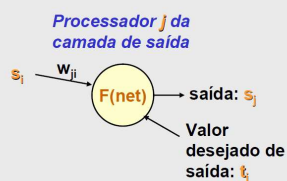
O algoritmo Back Propagation tem portanto duas fases, para cada padrão apresentado:

- Feed-Forward** → as **entradas** se propagam pela rede, da camada de entrada até a camada de saída.
- Feed-Backward** → os **erros** se propagam na **direção contrária ao fluxo de dados**, indo da camada de saída até a primeira camada escondida.

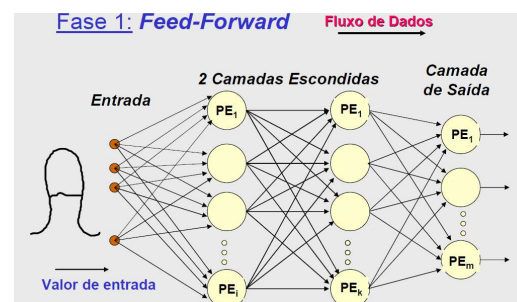
Cálculo do Erro (e_j)

❶ **Processador j pertence à Camada de Saída:**

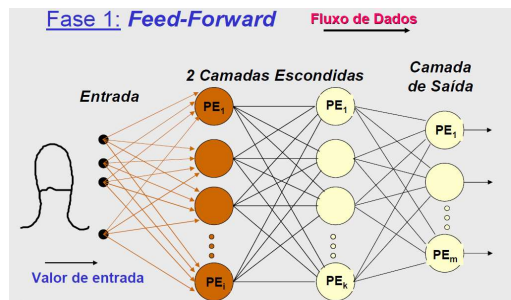
$$e_j = (t_j - s_j) \cdot F'(\text{net}_j)$$



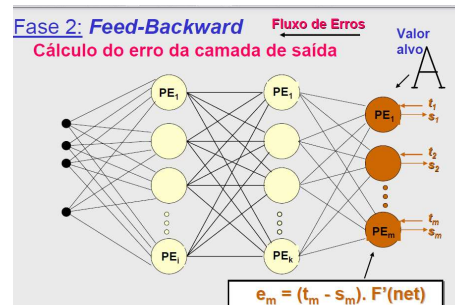
Processo de Aprendizado



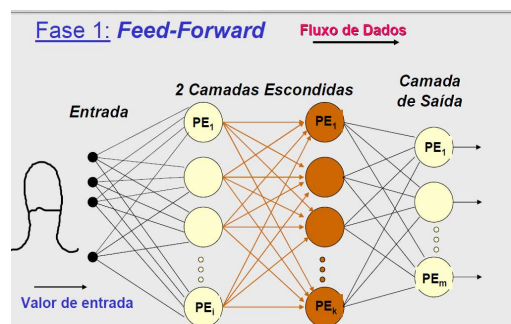
Processo de Aprendizado



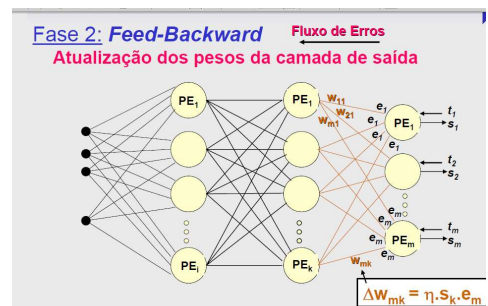
Processo de Aprendizado



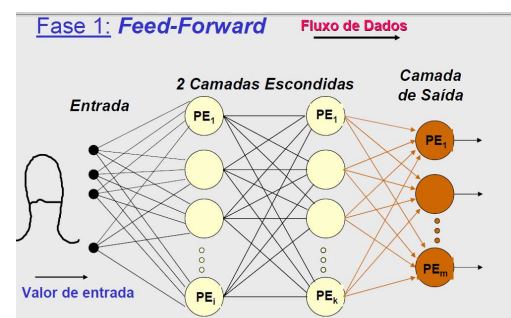
Processo de Aprendizado



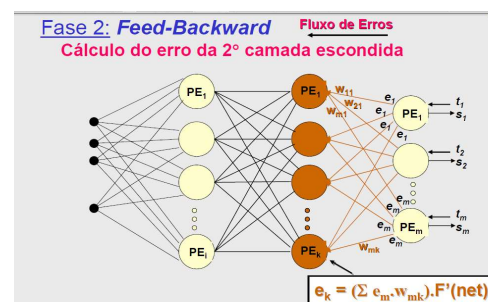
Processo de Aprendizado



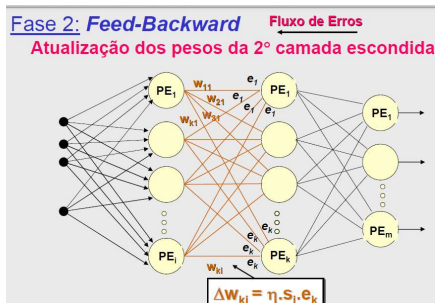
Processo de Aprendizado



Processo de Aprendizado



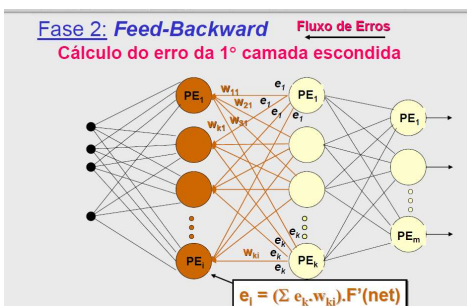
Processo de Aprendizado



Processo de Aprendizado

✳ Este procedimento de aprendizado é repetido diversas vezes, até que, para todos os processadores da camada de saída e para todos os padrões de treinamento, o erro seja menor do que o especificado.

Processo de Aprendizado



Algoritmo de Aprendizado

Inicialização:

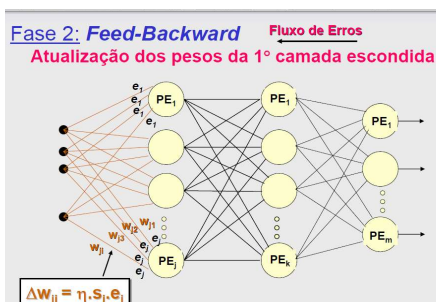
pesos iniciados com valores aleatórios e pequenos ($|w| \leq 0.1$)

Treinamento:

✳ Loop até que o **erro** de cada processador de saída seja $\leq$ **tolerância**, para **todos os padrões do conjunto de treinamento**.

- ❶ Aplica-se um **padrão de entrada X_i** com o respectivo **vetor de saída Y_i desejado**.
- ❷ Calcula-se as **saídas** dos processadores, começando da primeira camada escondida até a camada de saída.
- ❸ Calcula-se o **erro** para cada processador da camada de saída. Se **erro $\leq$ tolerância**, para todos os processadores, **volta ao passo ❶**.
- ❹ **Atualiza os pesos** de cada processador, começando pela camada de saída, até a primeira camada escondida.
- ❺ Volta ao passo ❶

Processo de Aprendizado



Avaliação do Algoritmo

- Foi demonstrado que a **Multi-Layer Perceptron** é um **Aproximador Universal**, isto é, pode representar qualquer função.



- O BP é o **algoritmo** de Redes Neurais **mais utilizado** em aplicações práticas de **previsão**, **classificação** e **reconhecimento de padrões** em geral

Capacidade das Multi-Layer Perceptrons

① Mostrou-se que 2 camadas escondidas são suficientes para **representar** regiões de qualquer tipo.

Cybenko 88 $\Rightarrow$ "Continuous valued neural networks with two hidden layers are sufficient", Technical report, Department of Computer Science, Tufts University, 1988.

Lippmann 87 $\Rightarrow$ "An Introduction to Computing with Neural Networks", ASSP Magazine, pp. 4-22, April 1987.

BP / Aproximador Universal

Teorema de Kolmogorov:

Qualquer mapeamento $f(x)$ contínuo de uma entrada p -dimensional ($p \geq 2$) em uma saída m -dimensional pode ser implementado **exatamente** por uma rede de p entradas, com uma camada escondida de $2p+1$ processadores e m processadores de saída.

$\Rightarrow$ Seja $f: \mathbb{I}^p \rightarrow \mathbb{R}^m$, $f(x) = y$

onde $\mathbb{I}^p$ é um hipercubo unitário p -dimensional

$$z_k = \sum_{j=1}^p \lambda^k \varphi(x_j + \varepsilon k) + k$$

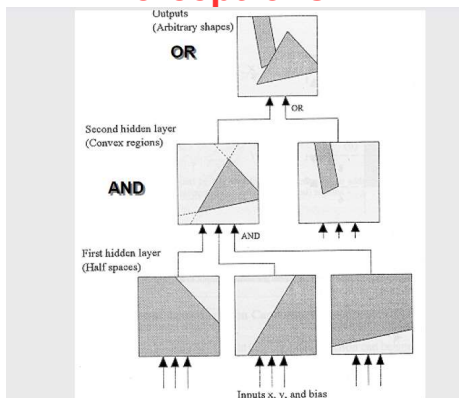
$$y_i = \sum_{k=1}^{2p+1} g_i(z_k)$$

onde: - λ é uma constante real;

- $\varphi(\cdot)$ é uma função monotônica crescente, contínua, real, independente de f mas dependente de p ;

- ε constante positiva.

Capacidade das Multi-Layer Perceptrons



BP / Aproximador Universal

Problemas:

$\Rightarrow$ como escolher os pesos λ ;

$\Rightarrow$ como definir as funções g_i , diferente para cada i , sendo geralmente de formato bem não-lineares;

$\Rightarrow$ as funções internas $\varphi(\cdot)$ são não suaves.

$\Rightarrow$ o teorema falha quando restrito a funções suaves.

Capacidade das Multi-Layer Perceptrons

② Demonstrou-se que as Multi-Layer Perceptrons com uma camada escondida são **Aproximadores Universais** (funções contínuas).

$\Rightarrow$ capacidade de aproximar, com precisão arbitrária, essencialmente **qualquer mapeamento contínuo do hipercubo $[-1, +1]$ no intervalo $(-1, +1)$** .

Cybenko 89 $\Rightarrow$ "Approximation by superpositions of a Sigmoidal Function", Mathematics of Control, Signals and Systems, 2, 303-314.

BP / Aproximador Universal

Teorema: Seja $\varphi(\cdot)$ uma função contínua, não-constante, limitada, monotônica crescente. Seja $\mathbb{I}_p$ o hipercubo unitário p -dimensional. O espaço de funções contínuas em $\mathbb{I}_p$ é $C(\mathbb{I}_p)$. Então, dada qualquer função contínua $f \in C(\mathbb{I}_p)$ e $\varepsilon > 0$, existe um inteiro M suficientemente grande, e um conjunto de constantes reais $\alpha_i, \theta_i, w_{ij}$ onde $i=1, \dots, M$ e $j=1, \dots, p$, tal que:

F é uma aproximação da função $f(\cdot)$:

$$F(x_1, \dots, x_p) = \sum_i \alpha_i \varphi(\sum_j w_{ij} x_j - \theta_i)$$

$$|F(x_1, \dots, x_p) - f(x_1, \dots, x_p)| < \varepsilon$$

Em termos de RNs: p = número de entradas
 M = número de PEs na camada escondida
 $\varphi(\cdot)$ = sigmóide na camada escondida
 função linear na saída

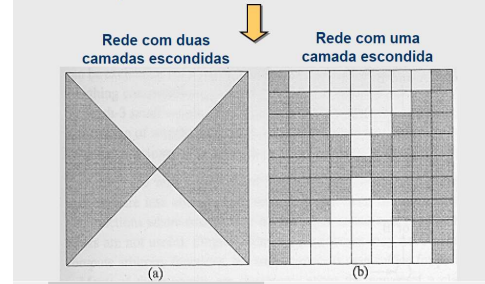
BP / Aproximador Universal

⇒ É somente uma prova de existência, não garantindo que:

- ⇒ uma única camada é ótimo em termos de tempo;
- ⇒ que é de fácil implementação;
- ⇒ nem que um determinado algoritmo de otimização irá encontrar a necessária configuração de pesos sinápticos.

BP / Aproximador Universal

⇒ Por que se utiliza mais de uma camada?



BP / Aproximador Universal

⇒ Verificou-se que o erro decresce na ordem de $O(1/\sqrt{N})$, conforme o número N de padrões aumenta.

⇒ Verificou-se também que o erro decresce na ordem de $O(1/M)$ em função do número M de processadores escondidos.

⇒ “Rule of Thumb” ⇒ $N > O(Mp / \varepsilon)$

onde N = # de padrões;

M = # de processadores escondidos;

p = dimensão da entrada ($Mp \approx$ # de parâmetros);

ε = erro mínimo desejado.

BP / Aproximador Universal

⇒ **Problema de múltiplas camadas intermediárias:**

⇒ cada vez que o erro medido é retro-propagado para a camada anterior, ele se torna **menos preciso**.

BP / Aproximador Universal

⇒ Por que se utiliza mais de uma camada?



⇒ A declaração que se necessita de um número “**suficientemente grande**” de processadores na camada escondida.

⇒ Quando $f(x)$ a ser aproximada é **não contínua**.

⇒ O número de processadores na camada única pode tender a infinito.

Fim do Back Propagation